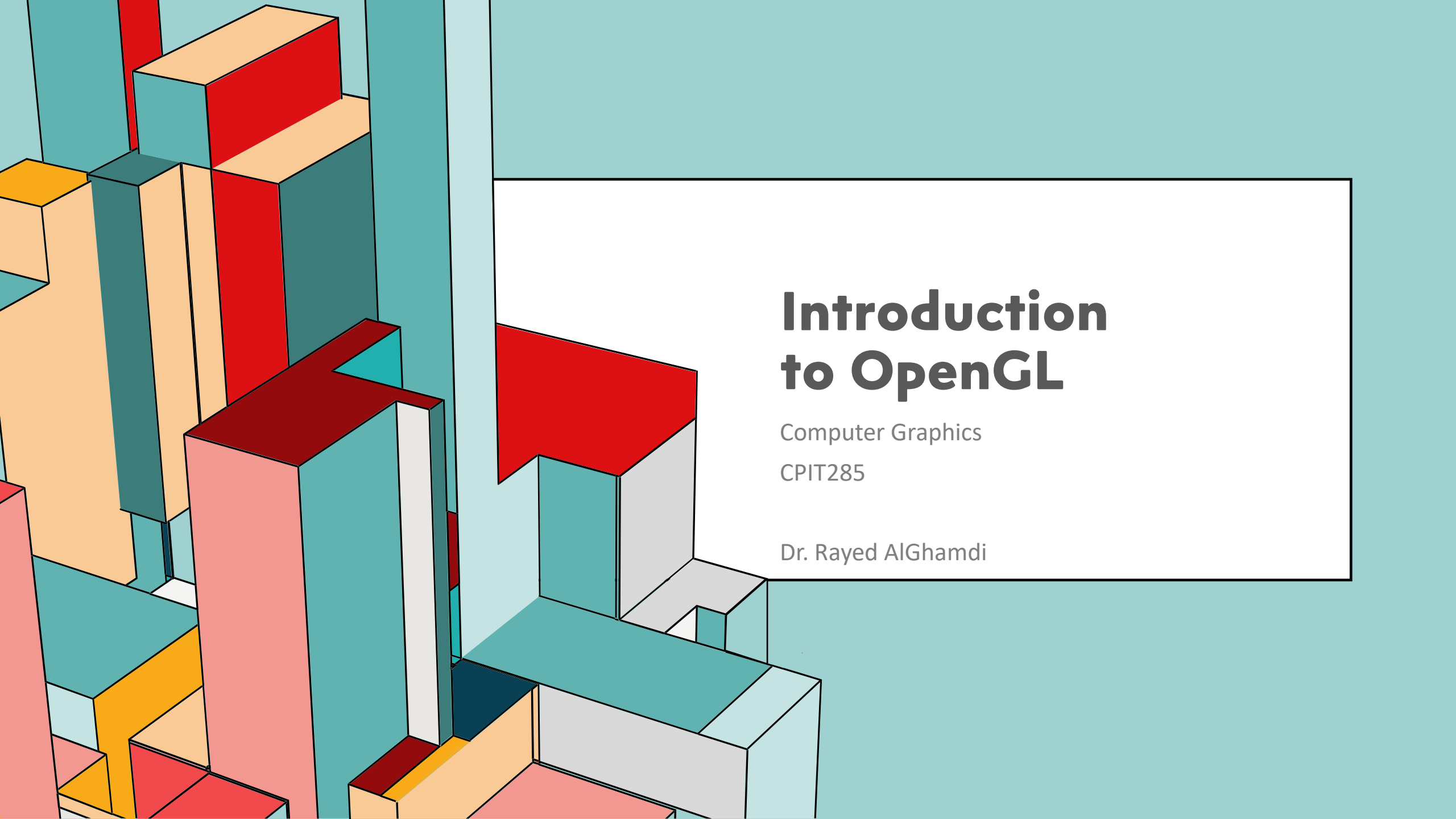




Introduction to OpenGL

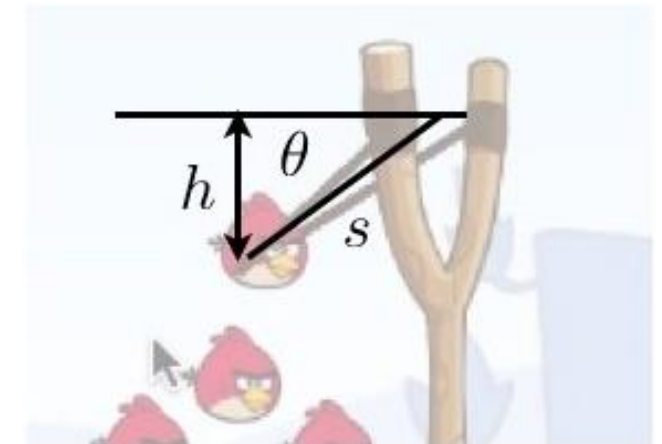
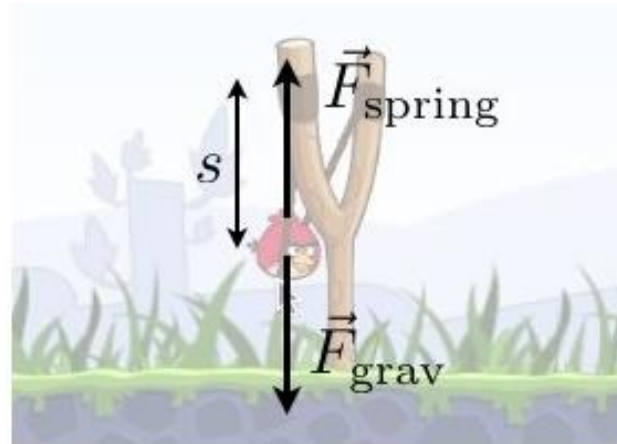
Computer Graphics

CPIT285

Dr. Rayed AlGhamdi

PREFACE

- In computer games we do not design fancy objects for the user to watch, they need to interact.
- The games need to receive user inputs, analyze them and make all changes according to the orders given...
- This process involves math calculation and physics simulation...



PREFACE

- In order for the game to be seen, there must be images representing the objects to be executed.
- These images need to be created and have a very quick appearance based on the game's data (user inputs) to show the game as a video clip or animated objects rather than a series of images displayed in a sequence.
- An experienced user may code algorithms to perform all physical logic, math calculations, manage the memory...
- however, how the drawing process can be handled?



PREFACE

- Early ages of desktop computers, drawing was very simple and the CPU managed the screen contents.
- E.g. drawing a line segment → CPU would run a loop to set the color of each pixel that lies along the line.
- Drawing process could take up a lot of the CPU's time.
- Graphics performance was very slow, compared to what we expect today.



PREFACE



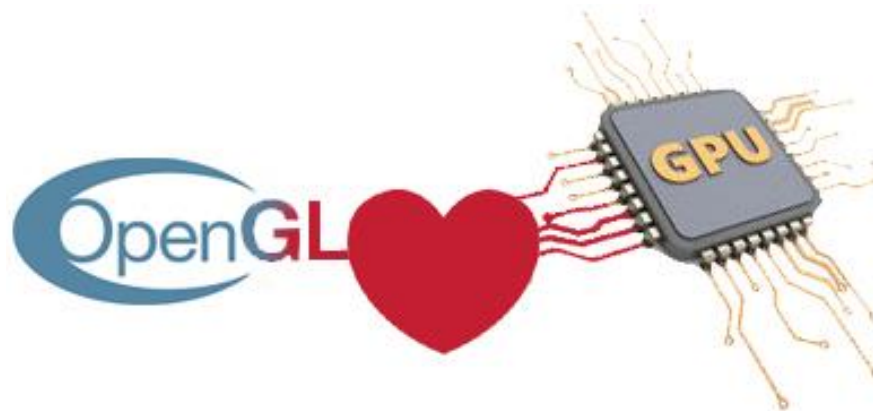
So what has changed?

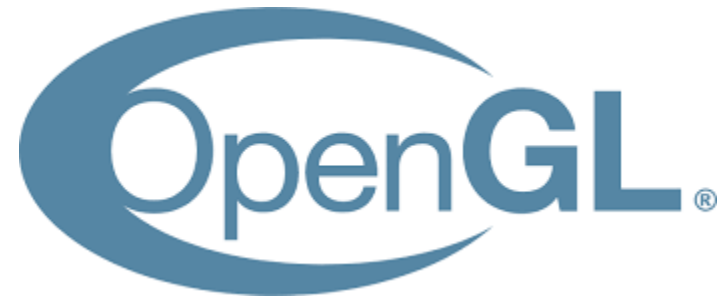
- Computers are much faster in general, of course, but the big change is that in modern computers, graphics processing is done by a specialized component called a GPU, or Graphics Processing Unit.
- To draw graphical operation:
 - CPU simply has to send commands, along with any necessary data, to the GPU,
 - GPU is responsible for actually carrying out those commands.
 - The set of commands that the GPU understands make up the API of the GPU.
- OpenGL is an example of a graphics API, and most GPUs support OpenGL in the sense that they can understand OpenGL commands, or at least that OpenGL commands can efficiently be translated into commands that the GPU can understand.

PREFACE

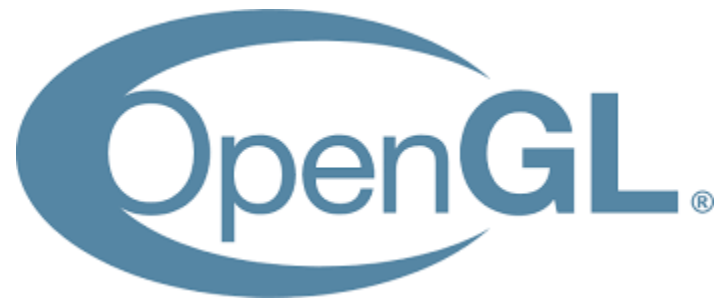
So what has changed?

- Back to the game, giving the CPU the role of drawings processes takes a long time because it has several tasks to manage at the same time in order to run the entire computer, thus the games will be very slow!
- To overcome this problem, [Open Graphical Library](#) (OpenGL) facilitates and accelerates these processes by direct access to the computer [Graphics Card](#)

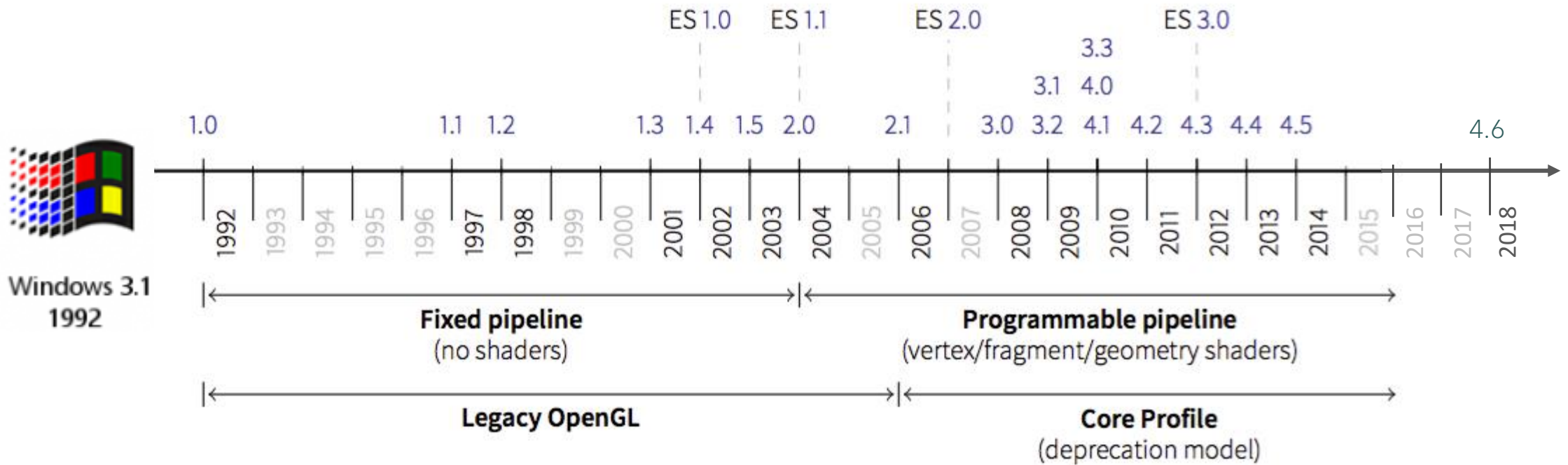




- The current version, in early 2018, is 4.6, and it is very different from the 1.0 version from 1992.
- In this course, we will begin our study of OpenGL with version 1.1. Most of the concepts and many of the details from that version are still relevant and functioning, and it offers an easier entry point for someone new to 3D graphics programming.
 - Furthermore, there is a specialized version called  for “embedded systems” such as mobile phones and tablets.
 - And there is also , for use in Web browsers.



Who creates and maintain... ?





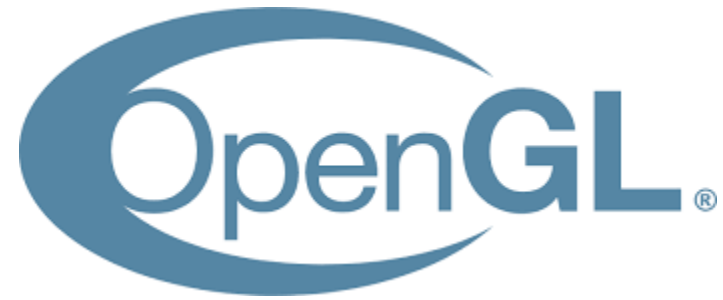
Getting Started...

- To start taking advantage of the OpenGL API for game development or software uses OpenGL, you need to install the appropriate graphics card driver which enables usage of the functionality provided.
- To program using the OpenGL API, you need the driver and the development package (depends on platform and programming language).
- In all three major desktop platforms (Linux, macOS, and Windows), OpenGL more or less comes with the system. However, you will need to ensure that you have downloaded and installed a recent driver for your graphics hardware.



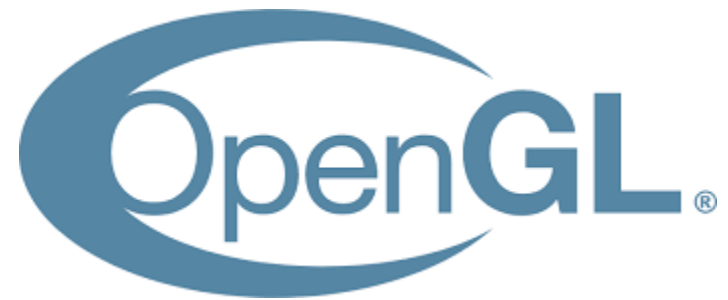
Writing an OpenGL Application

- The first step is to pick your programming language.
- Bindings for OpenGL exist in various programming languages: C#, C++, Java, Python...
- Some programming languages have multiple sets of OpenGL bindings, none of them being official. All of them are ultimately based on the C/C++ bindings.
- If you are not using C/C++, you need to install a package or library for your chosen language that includes the OpenGL bindings. Some come pre-installed, but others have separate downloads.



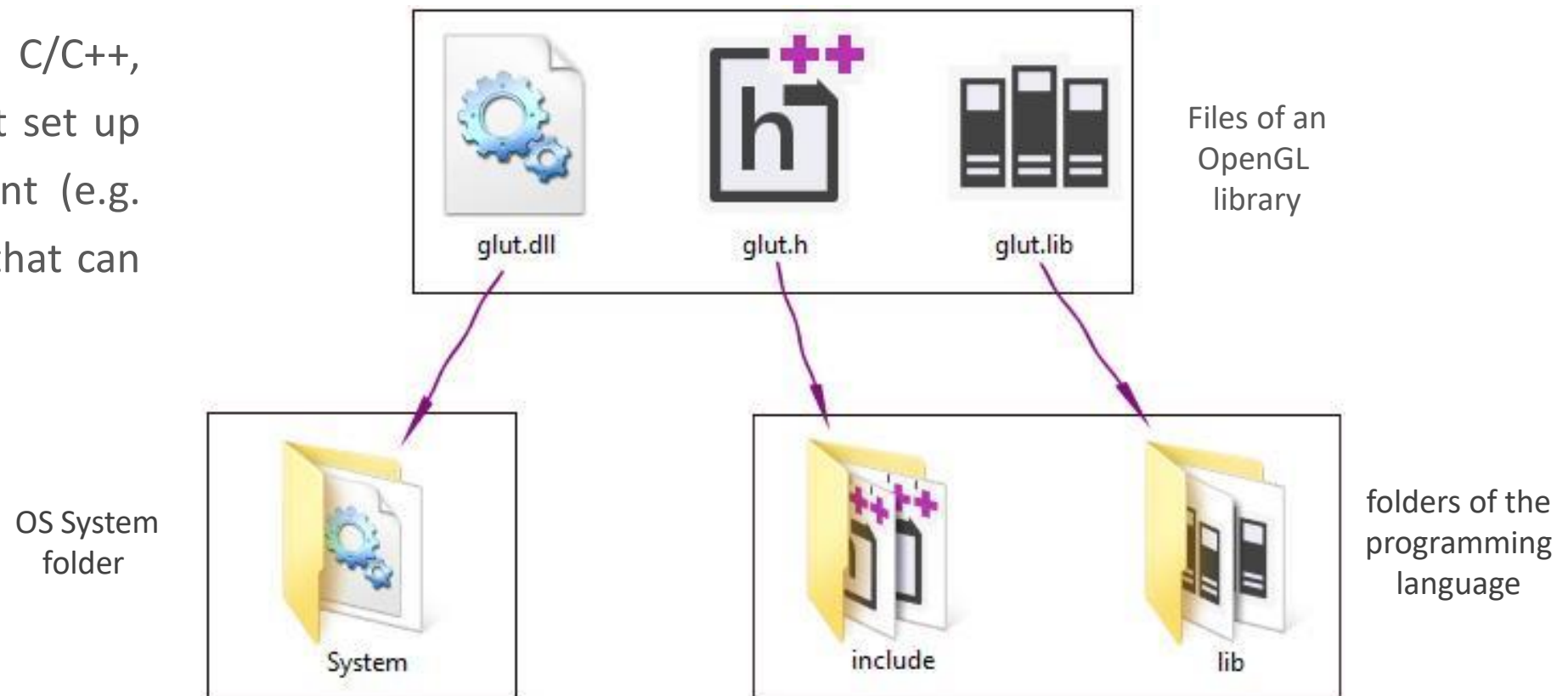
OpenGL Libraries

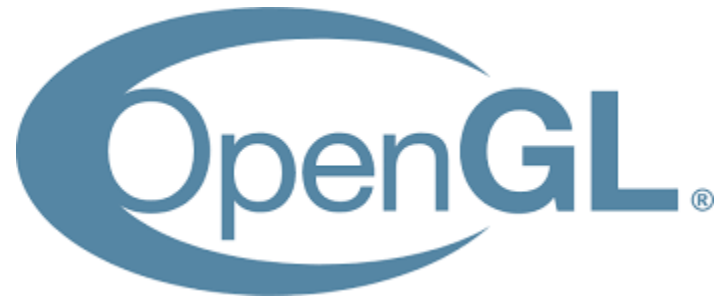
OpenGL Library	Functions
GL	handling the basics and initial drawing processes
GLU (Graphics Library Utilities)	advanced drawing, perspective drawing and some 3D shapes.
GLUT (Graphics Library Utilities Toolkit)	managing windows, interactive functions to receive user inputs.
GLUI (Graphics Library User Interface)	based on GLUT to create objects on the window such as a button, a text box, and so on.
GLEW / GLX / AUX	



Setting up Libraries

- If you are using C/C++, then you must first set up a build environment (e.g. MS Visual Studio) that can link to OpenGL.





Get Your Computer Ready!

- In this course, we use Python to write OpenGL applications.
- In your KAU foundation year, you were introduced to programming in Python.
- It is assumed that you are familiar with this language, and you understand its basics.
- Let's go with the following steps to get things ready writing our first OpenGL application using Python.

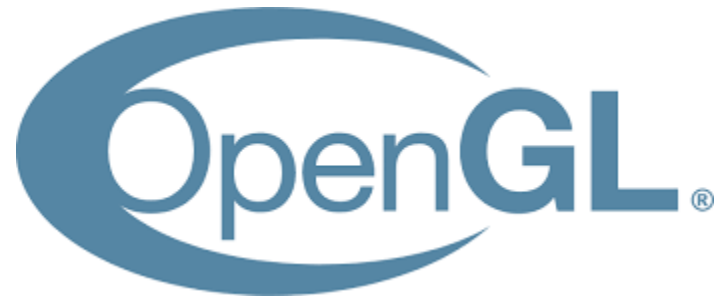




Get Your Computer Ready!

1. Install Python:

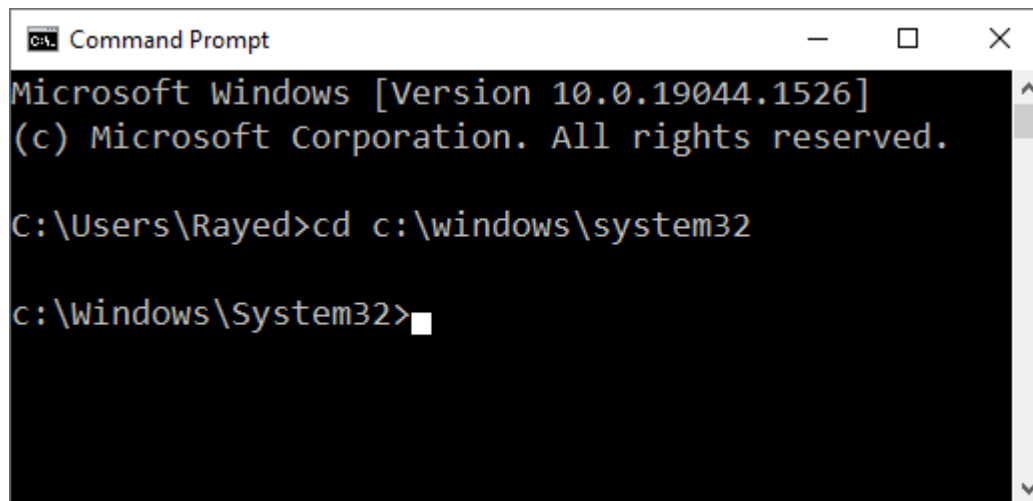
- Visit www.python.org to download the latest version 3.10
- Include Python IDLE (Integrated Development and Learning Environment).
- If you wish to use another IDLE, [PyCharm](#) is recommended.
- If you are a regular programmer using other languages, you may go with [Anaconda](#) or [MS Visual Studio](#) to support multi-languages.
- If you want to have a light app for viewing codes in a well-organized way, you may go with [Notepad++](#) or [sublime](#).
- To refresh your information on how to use Python, visit docs.python.org/3/tutorial/index.html



Get Your Computer Ready!

2. Install Python packages that allow using graphics functions:

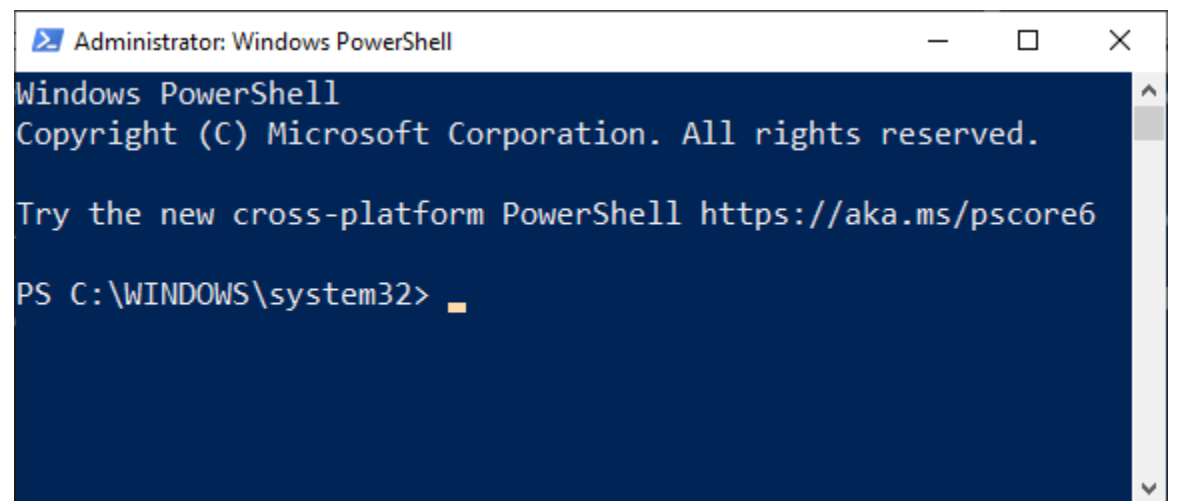
- Run command prompt or Windows PowerShell on your computer.
- Make sure to run with administrator privileges to install the packages for all users.

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt". The text inside shows the user navigating to the system32 directory. The prompt is at the c:\Windows\System32 directory.

```
Microsoft Windows [Version 10.0.19044.1526]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Rayed>cd c:\windows\system32

c:\Windows\System32>
```

A screenshot of a Windows PowerShell window running as an administrator. The title bar reads "Administrator: Windows PowerShell". The text inside shows the PowerShell prompt at the C:\WINDOWS\system32 directory.

```
Administrator: Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\WINDOWS\system32>
```



Get Your Computer Ready!

2. Install Python packages that allow using graphics functions:

- We need to install these 4 packages: pygame, numpy, PyOpenGL, PyOpenGL_accelerate

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\WINDOWS\system32> py -m pip install pygame numpy PyOpenGL
```

`py -m pip install pygame numpy PyOpenGL`

- When you get an error for installing PyOpenGL_accelerate
 - Visit [this page](#) to download PyOpenGL_accelerate-3.1.5-cp310-cp310-win_amd64.whl
- If you are using windows 64bit, then run the following command using the path where you install the above file.
- ```
py -m pip install c:\...\download\PyOpenGL_accelerate-3.1.5-cp310...
```





## Get Your Computer Ready!

### 2. Install Python packages\* that allow using graphics functions:

- **Pygame** is a “cross-platform set of Python modules\* designed for writing video games. It includes computer graphics and sound libraries designed to be used with the Python programming language”.
- **NumPy** is a “library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays”.
- **PyOpenGL** is a Python binding\* that provides access to the needed functions and constants through the OpenGL package.
- **PyOpenGL\_accelerate** consist of Cython\* accelerator modules that attempt to speed up various aspects of PyOpenGL

A **package** is a collection of related modules. A **module** is a file containing Python code. A **binding** is a wrapper library that bridges two programming languages so that a library written for one language can be used in another language. **Cython** is a programming language that aims to be a superset of Python, designed to give C-like performance with code that is written mostly in Python with optional additional C-inspired syntax.



Get Your Computer Ready!

### 3. Get the Python source code for several OpenGL examples

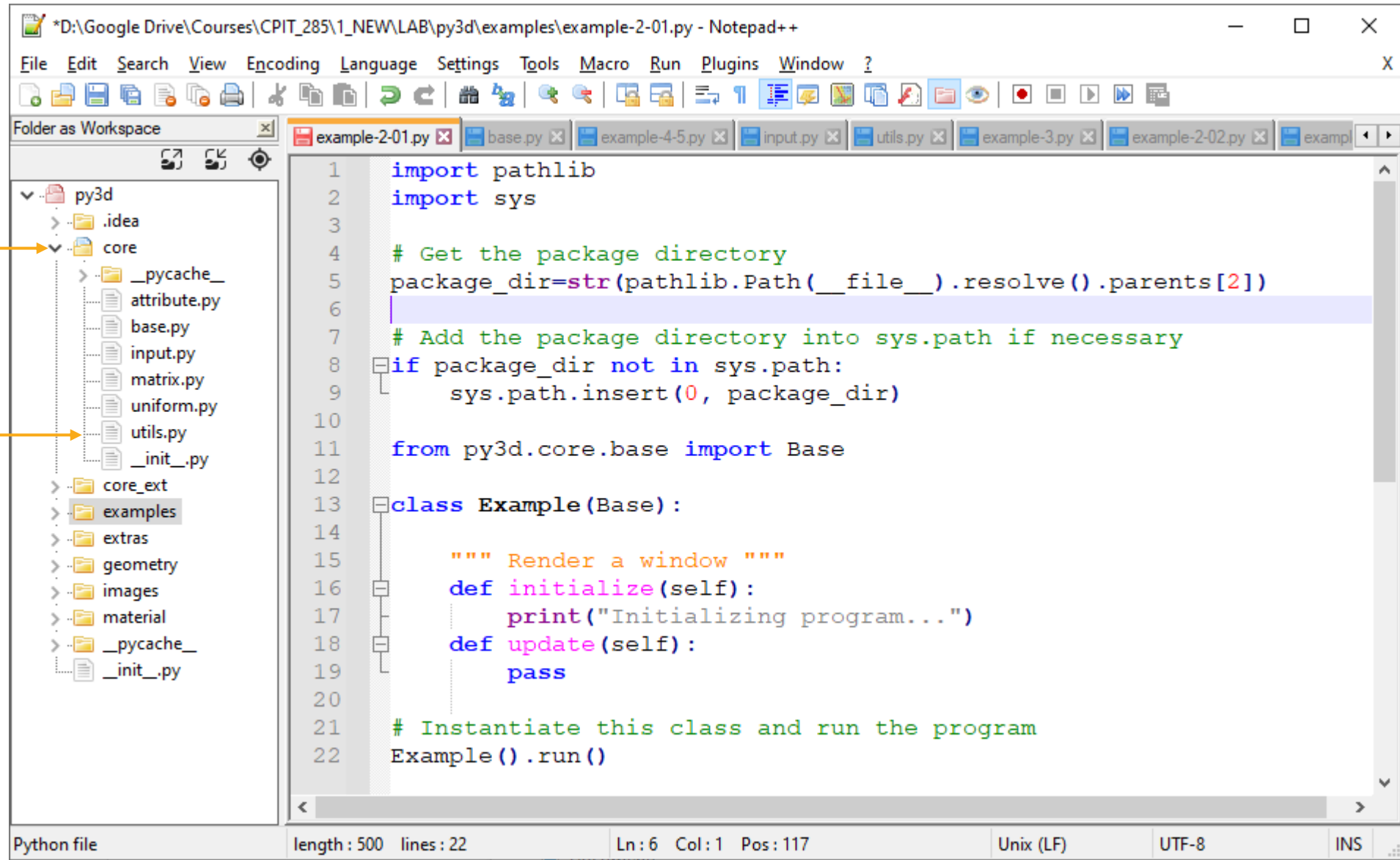
- Download from [GitHub](#)
- Unzip the folder and you should find the needed files in the “py3d” directory, well structured and organized.
- The files that can be executed are located in the “examples” directory.
- While we are progressing in this course, we keep referring to these examples to relate the theoretical concepts with practice.
- If you want to go step by step to building the code and organizing the files yourself from scratch, please follow the well-explained tutorials on [this playlist](#).

## Package

A package is a collection of related modules.

## Module

A module is a file containing Python code.



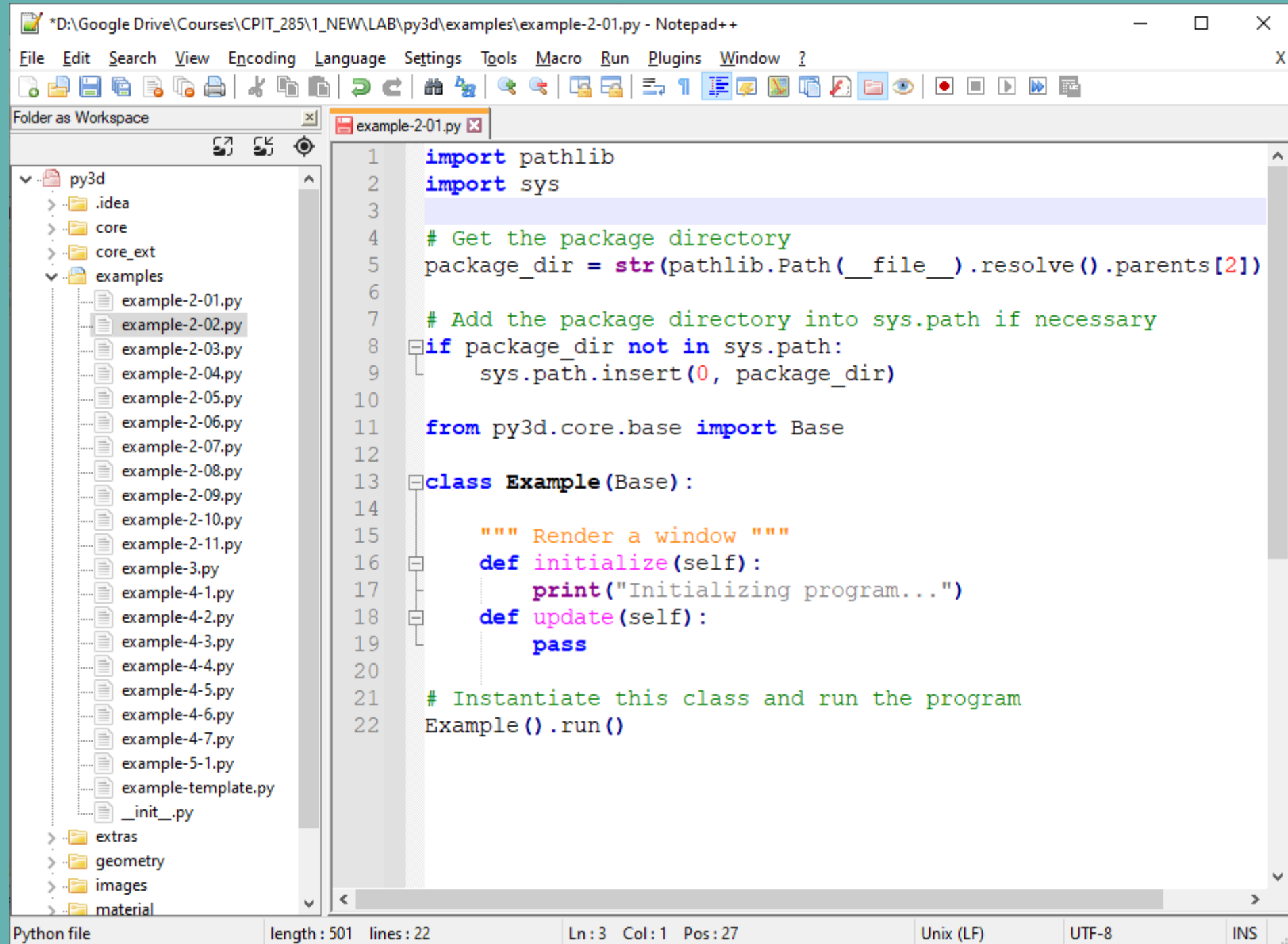
The screenshot shows a Notepad++ window with the file path `*D:\Google Drive\Courses\CPIT_285\1_NEW\LAB\py3d\examples\example-2-01.py`. The left sidebar displays a folder tree for the `py3d` package, which includes subfolders like `.idea`, `core`, `core_ext`, and `examples`, along with various Python files. An orange arrow points from the 'Package' definition to the `py3d` folder, and another points from the 'Module' definition to the `base.py` file within the `core` folder. The main editor area shows the code for `example-2-01.py`, which imports `pathlib` and `sys`, determines the package directory, adds it to `sys.path` if necessary, imports `Base` from `py3d.core.base`, and defines an `Example` class that inherits from `Base`. The class has an `initialize` method that prints a message and an `update` method that does nothing. Finally, an instance of `Example` is created and its `run` method is called.

```
1 import pathlib
2 import sys
3
4 # Get the package directory
5 package_dir=str(pathlib.Path(__file__).resolve().parents[2])
6
7 # Add the package directory into sys.path if necessary
8 if package_dir not in sys.path:
9 sys.path.insert(0, package_dir)
10
11 from py3d.core.base import Base
12
13 class Example(Base):
14
15 """ Render a window """
16 def initialize(self):
17 print("Initializing program...")
18 def update(self):
19 pass
20
21 # Instantiate this class and run the program
22 Example().run()
```

Python file length: 500 lines: 22 Ln: 6 Col: 1 Pos: 117 Unix (LF) UTF-8 INS

When you get the source code from GitHub, run the different files in the “examples” directory to view the output of each example.

Through our progress in this course, we keep referring to these examples to relate the theoretical concepts with practice.



```
*D:\Google Drive\Courses\CPIT_285\1_NEW\LAB\py3d\examples\example-2-01.py - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
Folder as Workspace
py3d
├── .idea
├── core
├── core_ext
├── examples
│ ├── example-2-01.py
│ ├── example-2-02.py
│ ├── example-2-03.py
│ ├── example-2-04.py
│ ├── example-2-05.py
│ ├── example-2-06.py
│ ├── example-2-07.py
│ ├── example-2-08.py
│ ├── example-2-09.py
│ ├── example-2-10.py
│ ├── example-2-11.py
│ ├── example-3.py
│ ├── example-4-1.py
│ ├── example-4-2.py
│ ├── example-4-3.py
│ ├── example-4-4.py
│ ├── example-4-5.py
│ ├── example-4-6.py
│ ├── example-4-7.py
│ ├── example-5-1.py
│ ├── example-template.py
│ └── __init__.py
├── extras
├── geometry
├── images
└── material

1 import pathlib
2 import sys
3
4 # Get the package directory
5 package_dir = str(pathlib.Path(__file__).resolve().parents[2])
6
7 # Add the package directory into sys.path if necessary
8 if package_dir not in sys.path:
9 sys.path.insert(0, package_dir)
10
11 from py3d.core.base import Base
12
13 class Example(Base):
14
15 """ Render a window """
16 def initialize(self):
17 print("Initializing program...")
18 def update(self):
19 pass
20
21 # Instantiate this class and run the program
22 Example().run()
```

Python file length : 501 lines : 22 Ln : 3 Col : 1 Pos : 27 Unix (LF) UTF-8 INS

# OpenGL Basic Syntax

- Functions starts with a prefix that expresses the library from which it's taken:
  - e.g. the function `glBindVertexArray` is taken from the GL library.
- When a function consists of more than one word, each word begins with a capital letter:
  - `glClearColor`
  - `glUseProgram`
- Constants start with the “GL”, followed by underscore ‘\_’ to split words, and all the letters in upper cases:
  - `GL_COLOR_BUFFER_BIT`
  - `GL_TRIANGLES`

# The Application Life Cycle

Every interactive graphics-based animated application required a set of stages.

- **Startup**

During this stage, objects are created, values are initialized, and any required external files are loaded.

- **The Main Loop**

This stage repeats continuously (typically 60 times per second), while the application is running and consists of the following three substages:

- **Process Input**

Check if the user has performed any action that sends data to the computer: e.g., keyboard or mouse click.

- **Update**

Changing values of variables and objects.

- **Render**

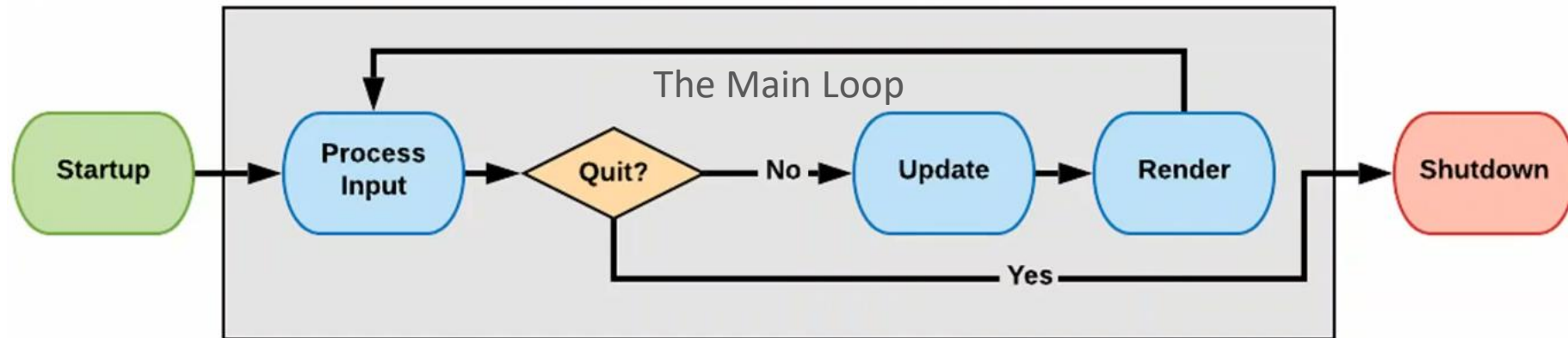
Create graphics that are displayed on the screen.

- **Shutdown**

It typically begins when the user performs an action indicating that the program should stop running.

# The Application Life Cycle

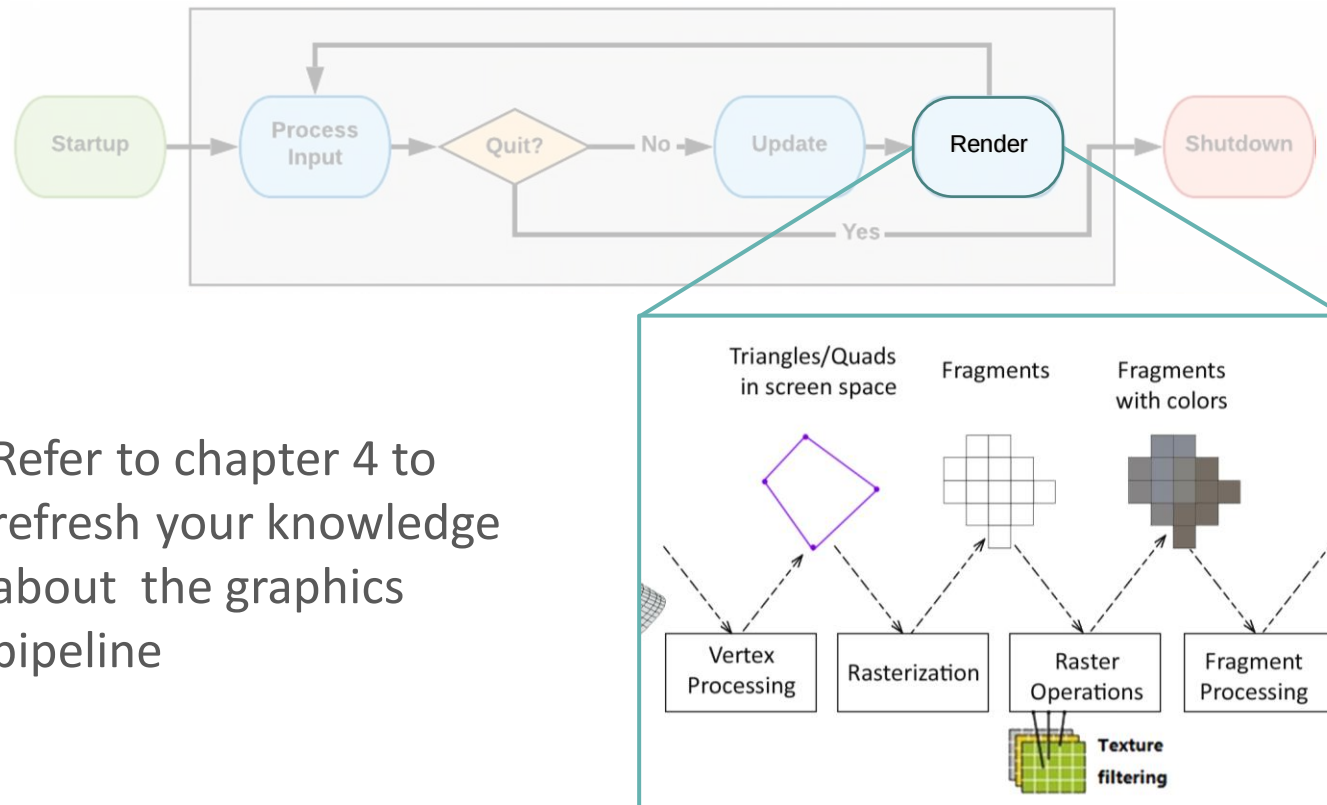
Every interactive graphics-based animated application required a set of stages.



The phases of an interactive graphics-based application

# The Application Life Cycle

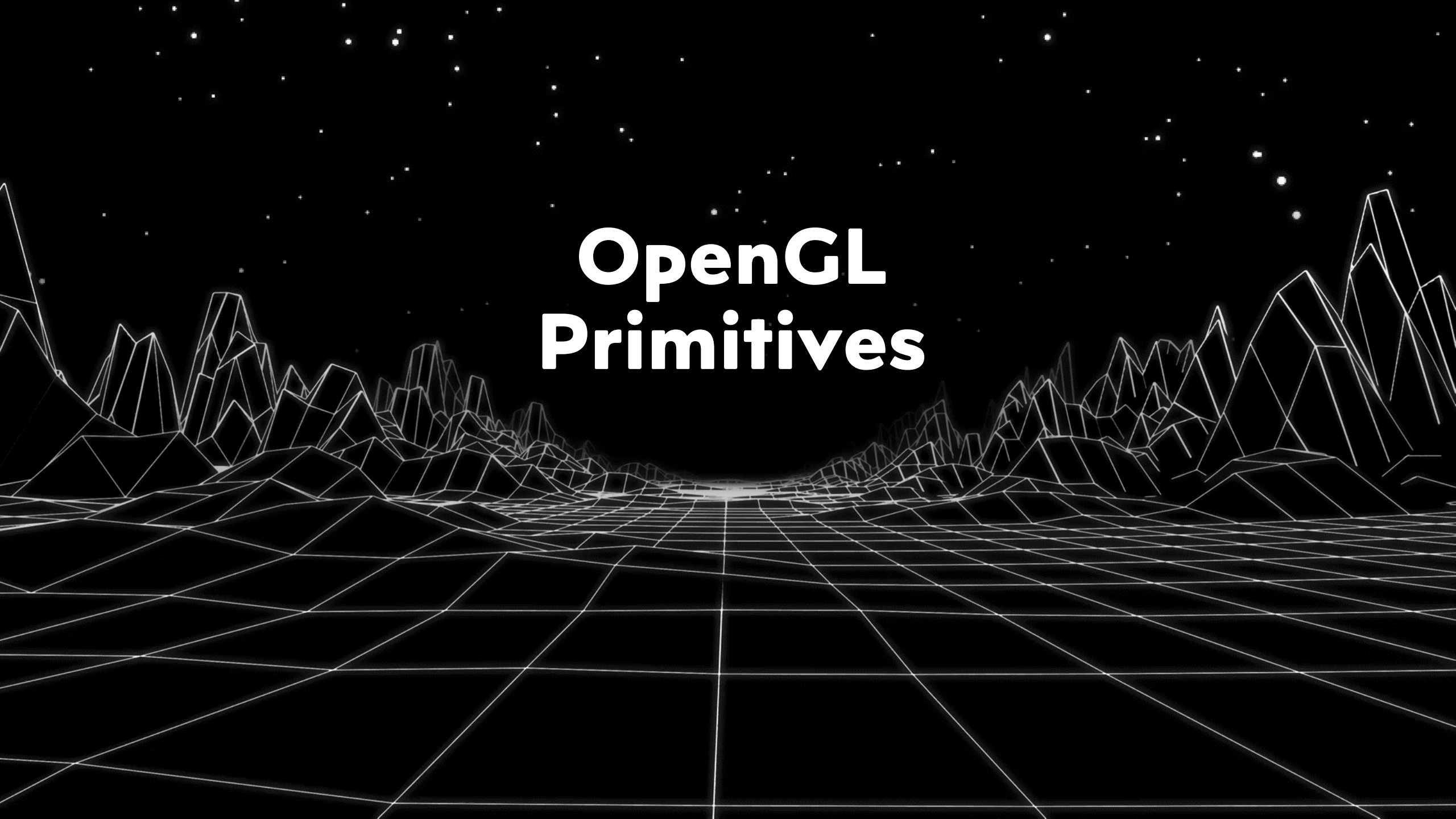
Every interactive graphics-based animated application required a set of stages.



Refer to chapter 4 to refresh your knowledge about the graphics pipeline

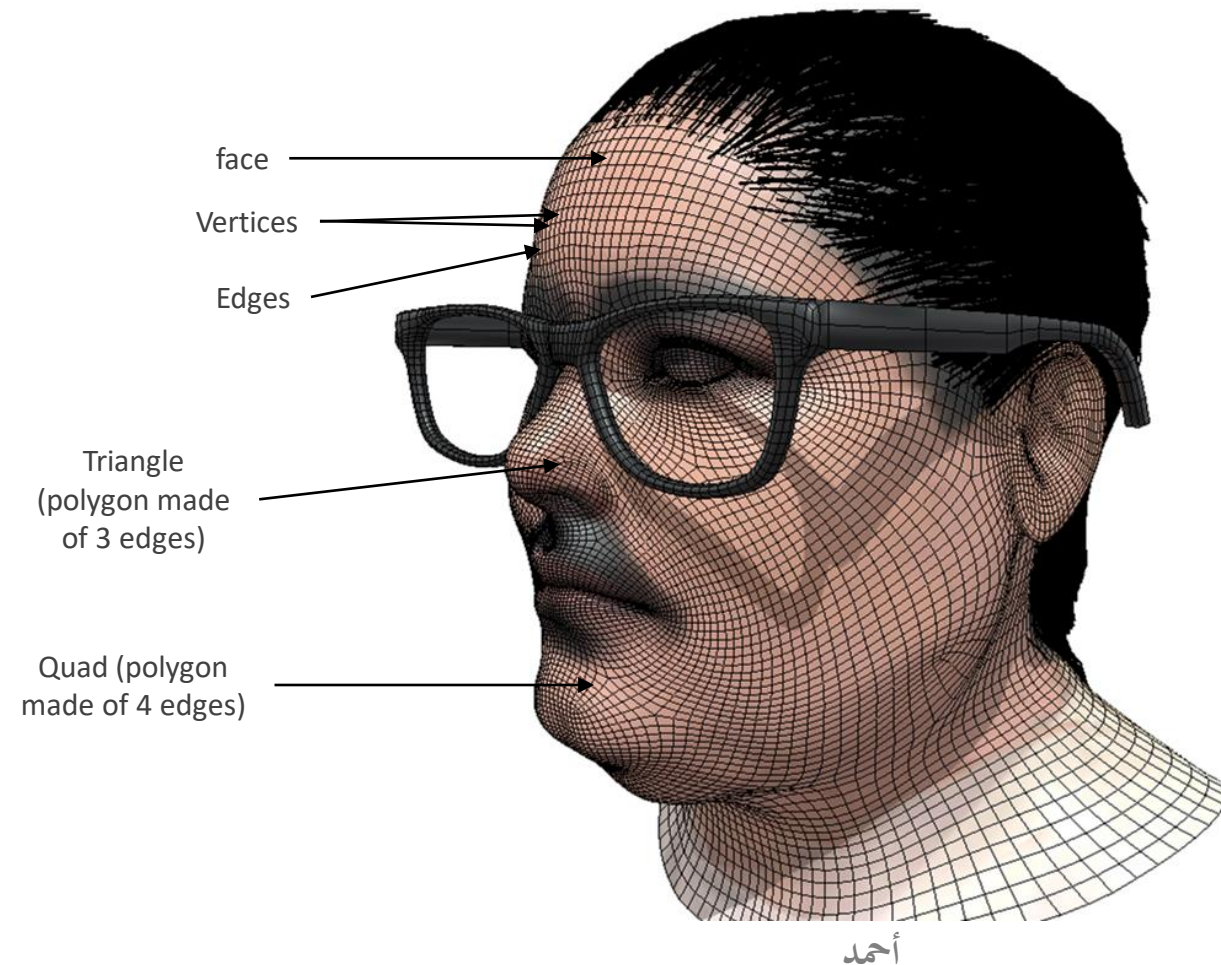


# OpenGL Primitives



# OpenGL Primitives

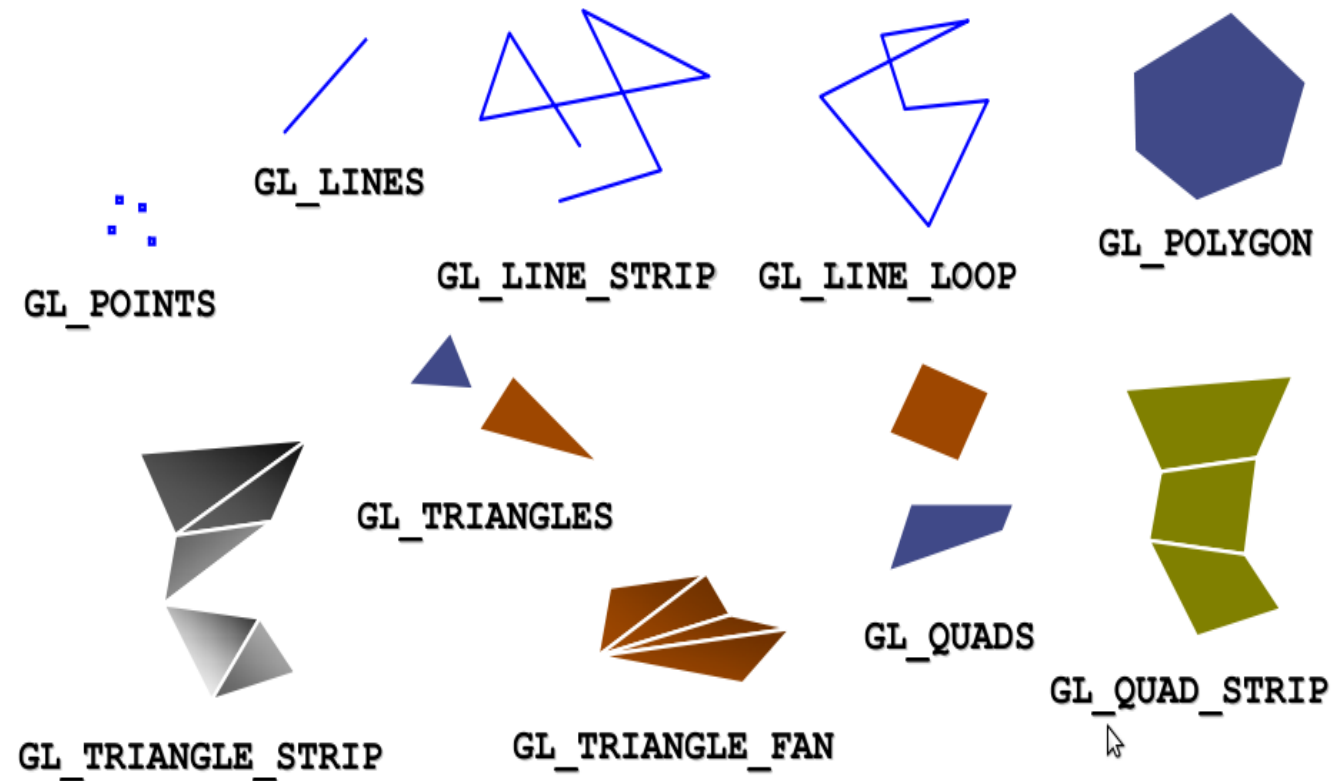
- In OpenGL, an object is made up of geometric primitives such as triangle, quad, line segment, and point.
- A primitive is made up of one or more vertices.
- What OpenGL does not do is retain information about an "object" as a whole!
- All OpenGL sees is a set of triangles/quads with which to render them.



# OpenGL Primitives

- OpenGL supports the following primitives
- All you need is to provide the position values for every vertex:

```
gl_Position =
vec4 (position.x, position.y, position.z, 1.0)
```



# OpenGL Primitives

| OpenGL Constants  | Role                                                                |                                                                                                                                                                                    |
|-------------------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GL_POINTS         | individual points                                                   |                                                                                                                                                                                    |
| GL_LINES          | pairs of vertices interpreted as individual line segments           |                                                                                                                                                                                    |
| GL_LINE_STRIP     | series of connected line segments                                   |                                                                                                                                                                                    |
| GL_LINE_LOOP      | same as above, with a segment added between last and first vertices |                                                                                                                                                                                    |
| GL_TRIANGLES      | triples of vertices interpreted as triangles                        |                                                                                                                                                                                    |
| GL_TRIANGLE_STRIP | linked strip of triangles                                           |                                                                                                                                                                                    |
| GL_TRIANGLE_FAN   | linked fan of triangles                                             |                                                                                                                                                                                    |
| GL_QUADS          | quadruples of vertices interpreted as four-sided polygons           | These functions have been removed from core OpenGL 3.1 and above (they are only deprecated in OpenGL 3.0). It is recommended that you not use this functionality in your programs. |
| GL_QUAD_STRIP     | linked strip of quadrilaterals                                      |                                                                                                                                                                                    |
| GL_POLYGON        | boundary of a simple, convex polygon                                |                                                                                                                                                                                    |

# OpenGL Primitives

## Point Function

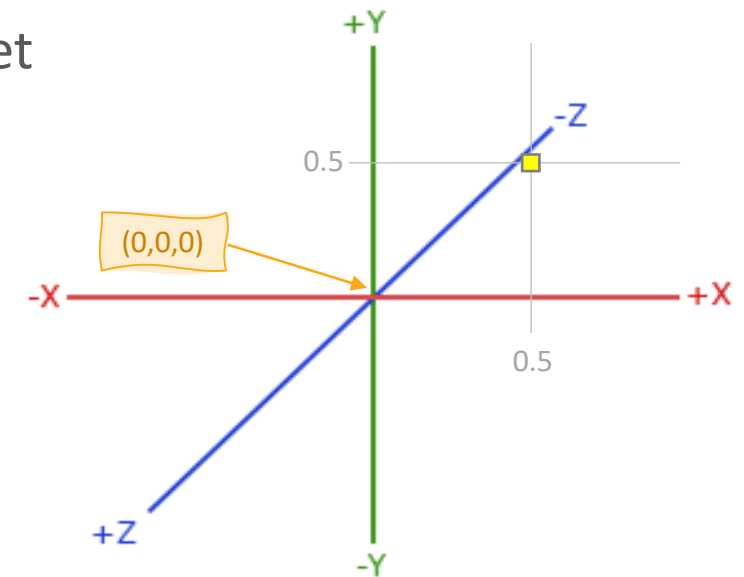
- The point has two attributes: size and color
- The default values: size = 1px, color = white
- In OpenGL, unless you change, the coordinates values are set by default -1 to +1 for every axis
  - What if I provide a position value that exceeds the default values?

```
GL.glPointSize(2)

fragColor = vec3 (1.0, 1.0, 0.0)

gl_Position = vec3 (0.5, 0.5, 0.0)

GL.glDrawArrays(GL_POINTS, 0, 1)
```



# OpenGL Primitives

## Line Functions

### ○ Three constants to draw lines in different ways:

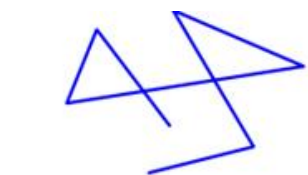
#### ■ `GL_LINES`

- Vertices 0 and 1 are considered a line. Vertices 2 and 3 are considered a line. And so on.
- If the user specifies a non-even number of vertices, then the extra vertex is ignored.



#### ■ `GL_LINE_STRIP`

- The adjacent vertices are considered lines. Thus, if you pass n vertices, you will get n-1 lines. If the user only specifies 1 vertex, the drawing command is ignored.



#### ■ `GL_LINE_LOOP`

- As line strips, except that the first and last vertices are also used as a line. Thus, you get n lines for n input vertices.
- If the user only specifies 1 vertex, the drawing command is ignored. The line between the first and last vertices happens after all of the previous lines in the sequence.

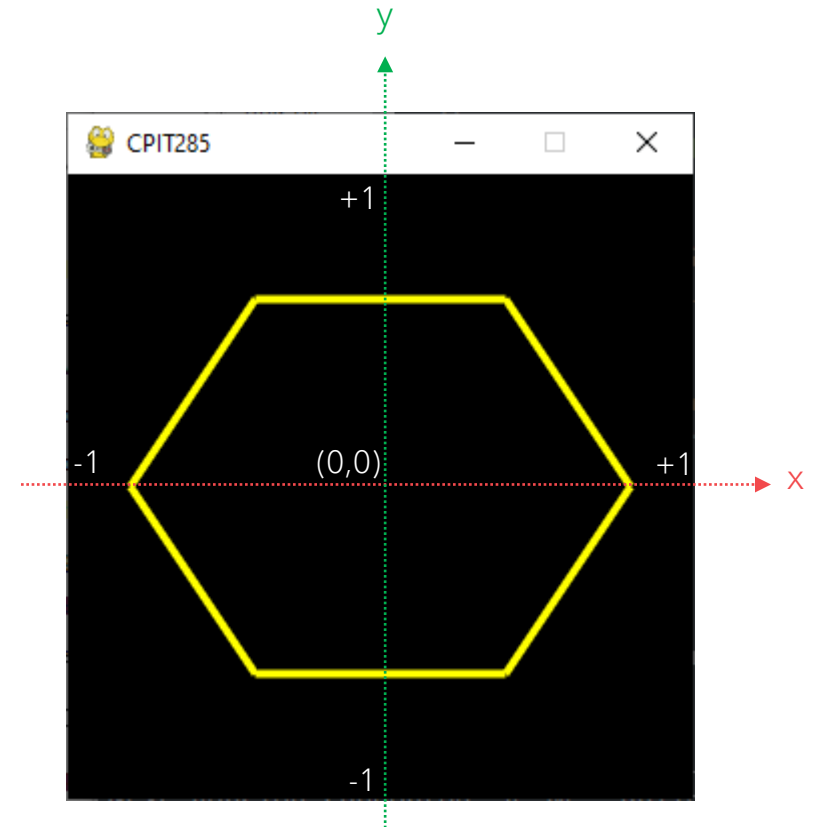


# OpenGL Primitives

## Line Functions

- Line attributes:
  - thickness (default = 1px)
  - color (default = white)
  - type (default = solid)

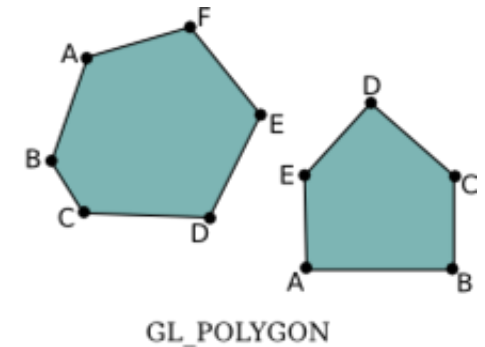
```
gl_Position = vec4(position.x, position.y, position.z, 1.0);
fragColor = vec4(1.0, 1.0, 0.0, 1.0);
GL.glLineWidth(4)
position_data = [[0.8, 0.0, 0.0],
 [0.4, 0.6, 0.0],
 [-0.4, 0.6, 0.0],
 [-0.8, 0.0, 0.0],
 [-0.4, -0.6, 0.0],
 [0.4, -0.6, 0.0]]
position_attribute = Attribute('vec3', position_data)
GL.glDrawArrays(GL.GL_LINE_LOOP, 0, self.vertex_count)
```



# OpenGL Primitives

## Polygon and Quad Functions

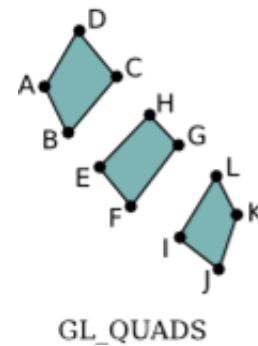
- A polygon is a closed filled area with three or more vertices.
  - The shape depends on the number of vertices
  - It can be a triangle, quad, pentagon, hexagon...



- A quad is a shape with four edges

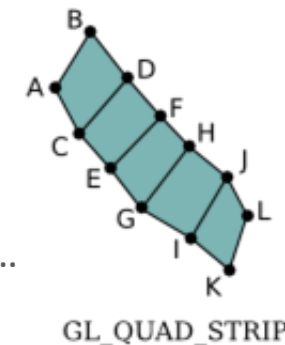
- **GL\_QUADS**

- connect every group of 4 vertices together
- what if the number of vertices is 6?



- **GL\_QUAD\_STRIP**

- The latest 2 vertices for a quad represent the first 2 vertices for the followed quad...

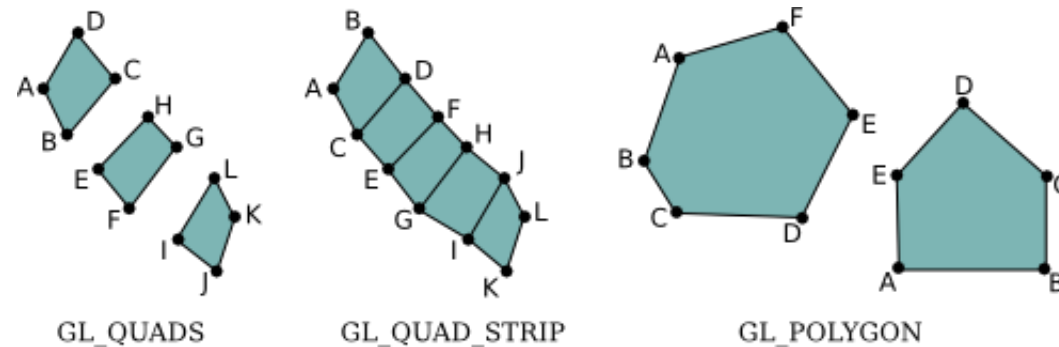




# OpenGL Primitives

## Polygon and Quad Functions

- The following primitives have been **removed** from modern OpenGL (3.1 and above):



- In order for a quad to be rendered correctly in OpenGL, all vertices of the quad must lie in the same plane. The same is true for polygon primitives.
- Since OpenGL doesn't check whether these conditions are satisfied, the use of quads and polygons is error-prone.

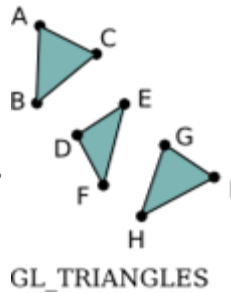
# OpenGL Primitives

## Triangle Functions

- Three OpenGL constants to draw triangles in different ways:

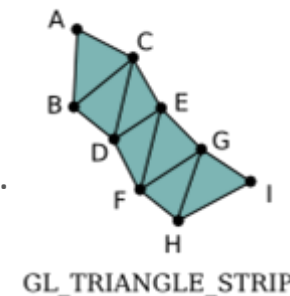
- **GL\_TRIANGLES**

- Using this function with a number of vertices not divisible by 3 will ignore the incomplete primitive at the end.
- Vertices 0, 1, and 2 forms a triangle. Vertices 3, 4, and 5 form a triangle. And so on.



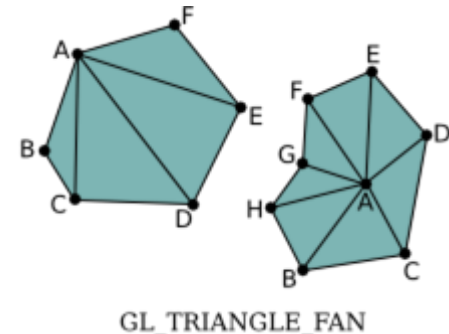
- **GL\_TRIANGLE\_STRIP**

- Every group of 3 adjacent vertices forms a triangle.
- The face direction of the strip is determined by the winding of the first triangle.
- A vertex stream of n length will generate n-2 triangles.



- **GL\_TRIANGLE\_FAN**

- The first vertex is held fixed.
- From there on, every group of 2 adjacent vertices forms a triangle with the first.
- A vertex stream of n length will generate n-2 triangles.

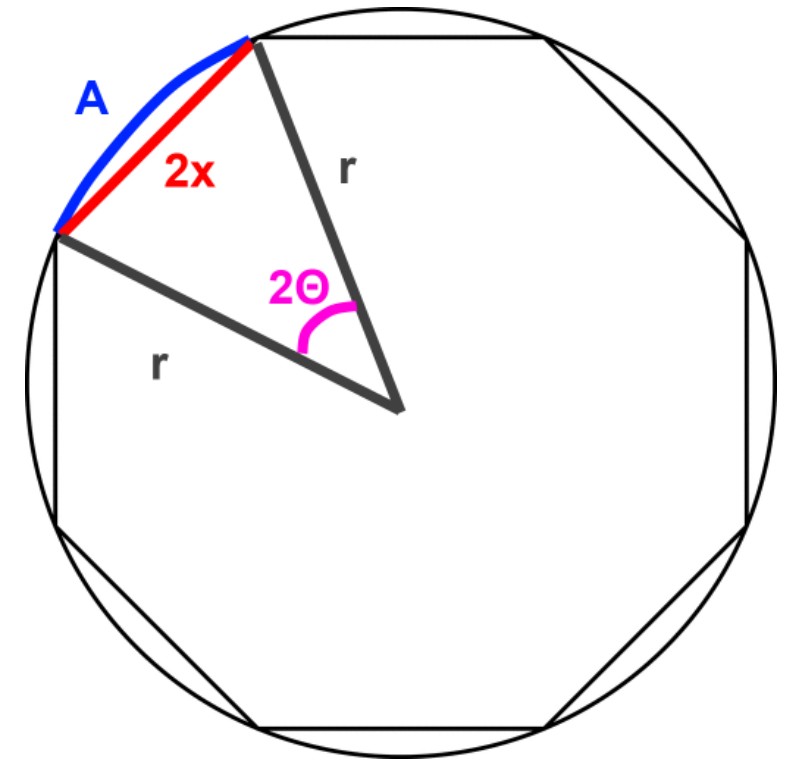


# OpenGL Primitives

## Is Circle Considered Primitive?

- OpenGL does not have a circle primitive.
- we can approximate a circle by drawing a polygon with a large number of sides.
- To draw an outline of the polygon, we can use a `GL_LINE_LOOP` primitive or `GL_TRIANGLE_FAN`.
- Keep in mind the circle mathematical concepts (radius, sin, cos, midpoint, PI, etc.)

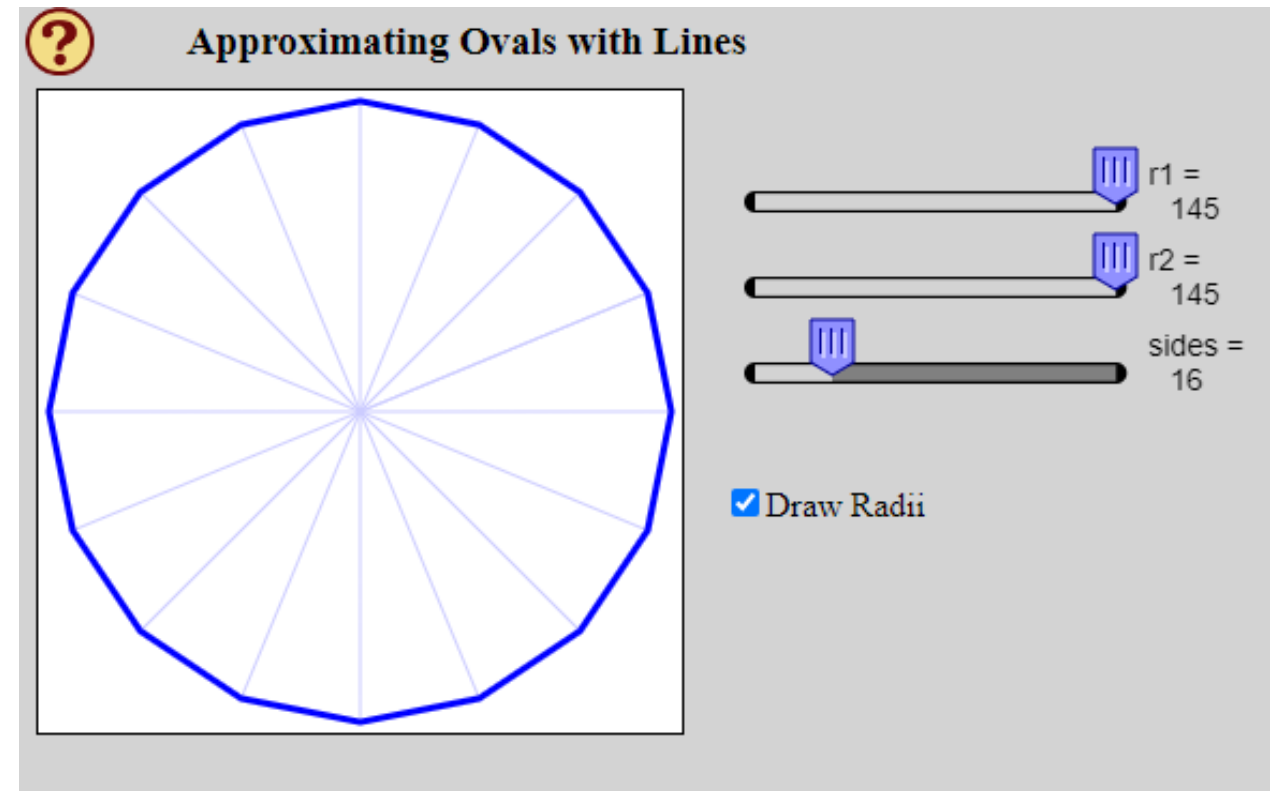
[Try this demo](#)



# OpenGL Primitives

## Is Circle Considered Primitive?

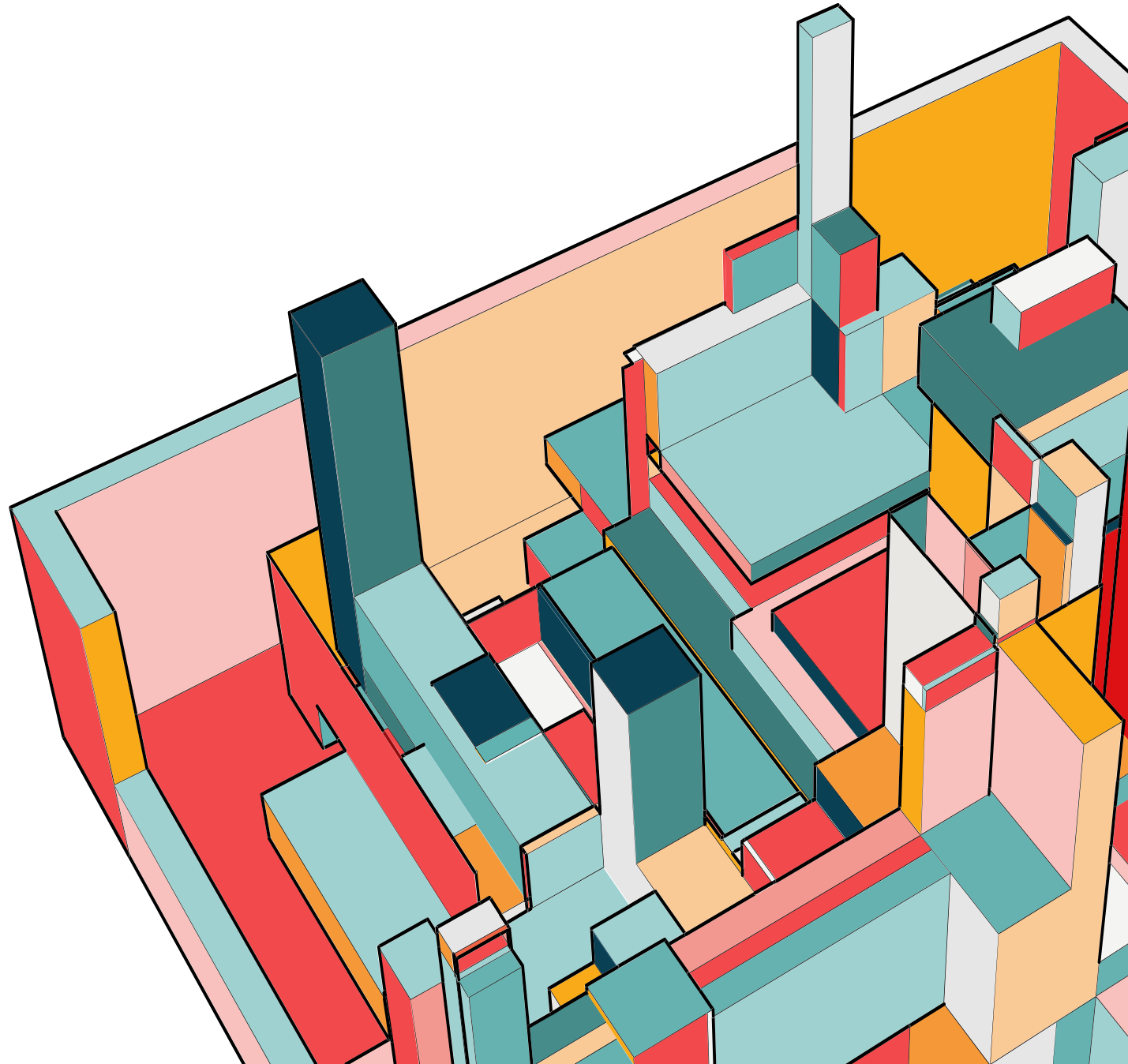
- The demo uses lines/triangles to approximate a circle.
- With a larger number of sides, you get a better approximation.
- The first two sliders control the size of the oval.
- The third controls the number of sides.
- Note how the number of sides that you need to get a good approximation depends on the size of the oval.



# 100% CONFIDENT

Your journey in this course is  
enjoyable and beneficial!

Enjoy learning CPIT285!



# THANK YOU

**Dr. Rayed ALGhamdi**

+966-12-640-0000 ex:75560

raalghamdi8@kau.edu.sa

[www.raalghamdi8.kau.edu.sa](http://www.raalghamdi8.kau.edu.sa)

